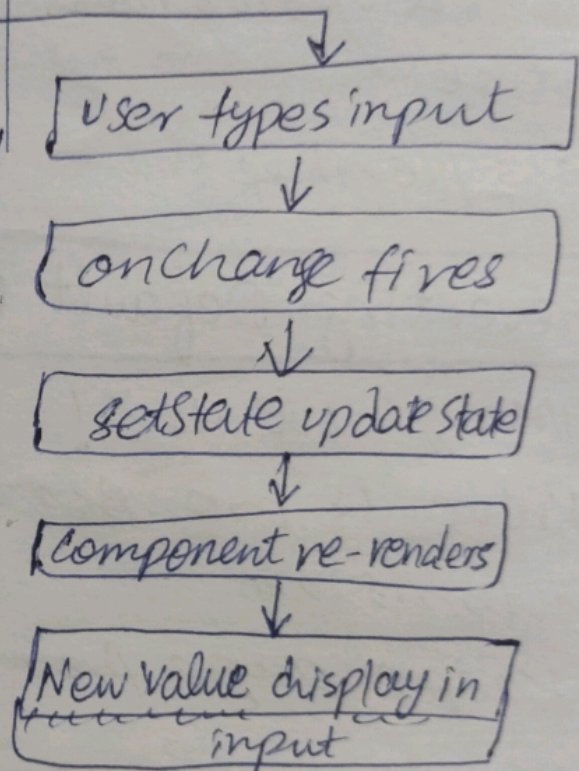


Controlled Component Uncontrolled Comp

- | | |
|--|---|
| → Value controlled by React | → Value controlled by DOM |
| → Use <code>value</code> & <code>onChange</code> props | → Uses <code>ref</code> to access value |
| → React is single source of truth. | → DOM is source of truth. |

Why this used?

- Can validate on every keystroke
- show/hide base on input values
- Easy to clear form by resetting react state.



Handle Multiple Inputs

Individual states

```
const [name, setName] = useState("");  
const [email, setEmail] = useState("");  
const [age, setAge] = useState("");
```

```
<input value={name} onChange={e =>  
  setName(e.target.value)} />  
...
```


Object State

```
function Object() {  
  const [formData, setFormData] = useState({  
    name: "",  
    email: "",  
    age: ""  
  });
```

```
  const handleChange = (e) => {  
    const {name, value} = e.target;  
    setFormData({  
      ...formData,  
      [name]: value  
    });  
  };  
};
```

```
return (  
  <form>  
    <input name="name" value={formData.name}  
      onChange={handleChange} />  
    :  
  </form>  
);
```


Forms in React & HTML

● HTML Forms (Traditional)

```
<form action="/submit" method="POST">
```

```
  <input type="text" name="username"/>
```

```
  <button type="submit">Submit</button>
```

```
</form>
```

- Form data managed by the DOM
- Submit cause page reload
- Direct access to form variable via `e.target.elem`

● React Forms

```
function MyForm() {
```

```
  const [username, setUsername] = useState("");
```

```
  const handleSubmit = (e) => {
```

```
    e.preventDefault();
```

```
    console.log(username);
```

```
  };
```

```
  return(
```

↙ on submit event

```
    <form onSubmit={handleSubmit}>
```

```
      <input type="text" value={username}
```

```
        onChange={e => setUsername(e.target.value)} />
```

```
      <button type="submit">Submit</button>
```


- Form data managed by React State
- No page reload
- Full control over form behavior.

HTML Form

- DOM manages data
- Page reload on submit
- Limited validation control
- less flexible

React Forms

- React state manages data
- No reload (SPA behavior)
- Full validation control
- Highly flexible

Preventing Default Behavior

Always use this!

- Prevents browser's default form submission
- Keeps page from reloading
- Maintain single-page app experience
- Allow React to handle submission

----- Without preventDefault(): -----

- Page will reload
- You'll lose react state
- Bad User experience

Form Validation

Required Field

```
const isValid = value.trim() !== "";
```

Email Validation

```
const isValidEmail = (email) => {  
  return /^[^\s@]+@[^\s@]+\.[^\s@]+  
    test(email);  
};
```

Min/Max length

```
const isValidLength = value.length >= 3  
  && value.length <= 20;
```

Number Range

```
const isValidAge = age > 0 && age < 150;
```

Showing Error

```
if (value === "") {  
  setError('Email is required');  
} else-if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(value)) {  
  setError('Email is invalid');  
} else {  
  // ...  
}
```


Manage Button Submit State

```
const isFormValid = email.includes('@') &&  
    password.length >= 6;
```

```
<button type="submit" disabled={!isFormValid}  
    </button>
```

Input Types:

Select Dropdown

```
const [country, setCountry] = useState("");
```

```
<select value={country} onChange=  
    {(e) => setCountry(e.target.value)} >
```

```
<option value="">Select Country</option>
```

```
<option value="usa">USA</option>
```

```
<option value="uk">UK</option>
```

```
</select>
```

Checkbox

```
const [agreed, setAgreed] = useState(false)
```

```
<input
```

```
    type="checkbox"
```

```
    checked={agreed}
```

```
    onChange={(e) => setAgreed(e.target.  
        checked)} >
```


Radio Button

```
const [gender, setGender] = useState("");
```

```
<input
```

```
  type="radio"
```

```
  name="gender"
```

```
  value="male"
```

```
  checked={gender === 'male'}
```

```
  onChange={(e) => setGender(e.target.value)}
```

```
/>
```

```
<input
```

```
  type="radio"
```

```
  name="gender"
```

```
  value="female"
```

```
  checked={gender === 'female'}
```

```
  onChange={(e) => setGender(e.target.value)}
```

```
/>
```


File Input

```
const [file, setFile] = useState(null);
```

```
<input
```

```
  type = "file"
```

```
  onChange = {(e) => setFile(e.target.files[0])}
```

How Validation Work?

- Store input value in state
- Check validation rule on change/submit
- Store error messages in state
- Display errors conditionally
- Disable submit if invalid